

Deep Learning — Spring 2026

Assignment 5

From Supervised Learning to Self-Supervised Learning

A Guided Implementation of SimCLR

Student: Fahad Khalid

Roll Number: MSDS25025

MS Data Science — 2025 Cohort

Information Technology University, Lahore

May 2026

Item	Value
Dataset	CIFAR-10
Random Seed	2026
Batch Size	64
SimCLR Epochs	50
Linear Probe Epochs	20
Fine-tuning Epochs	20
Optimizer	Adam
Learning Rate	3e-4
Temperature (tau)	0.5
Projection Dimension	128
GPU Used	Kaggle T4 x2
Training Time	~90 minutes total

Task 0: Conceptual Warm-up

Q1. What information does supervised classification learn from?

In supervised classification, the model learns from labeled data — pairs of input images and their corresponding class labels. The cross-entropy loss uses these labels to compute gradients and update model weights. The model learns to map visual features to output classes by minimizing prediction error against ground truth.

Q2. Do CIFAR-10 images still contain useful structure without labels?

Yes. The images contain rich visual structure — edges, textures, shapes, colors, and spatial patterns. The semantic content (airplanes, cars, dogs) does not disappear when labels are removed. Labels are human annotations; the underlying visual information is inherent to the images. Self-supervised learning exploits this structure without needing labels.

Q3. Should two augmented views of the same image have similar representations?

They should have similar representations. Since both views derive from the same original image, they contain the same semantic content. A good encoder should produce representations that capture this shared identity regardless of surface-level differences introduced by augmentation. SimCLR trains the model to achieve exactly this invariance.

Q4. Should views from different images have similar representations?

They should have different (dissimilar) representations. Different images typically contain different objects. If representations were similar, the model would not learn discriminative features. SimCLR pushes representations of different images apart in feature space, ensuring the encoder distinguishes between different visual inputs.

Q5. Why is a model trained without labels still useful for classification?

Because useful visual representations do not require labels to learn. When the encoder is trained to recognize that two views of the same image are similar, it must internally learn features like object shape, texture, and structure — exactly the features useful for classification. These representations can then be used with a simple linear classifier, requiring very few labeled examples.

Q6. What is the difference between pretraining and fine-tuning?

Pretraining trains a model on a large dataset (often without labels) to learn general-purpose feature representations — not optimized for any specific task. Fine-tuning takes the pretrained model and continues training on a smaller labeled dataset for a specific downstream task. Fine-

tuning adapts general representations to task-specific requirements, typically yielding better performance than training from scratch with limited labels.

Task 1: Supervised Baseline with Limited Labels

A ResNet-18 classifier was trained from scratch using only the fixed 10% labeled training split (5,000 images, 500 per class). This baseline establishes the performance ceiling of standard supervised learning under limited label conditions.

Training Configuration

Hyperparameter	Value
Architecture	ResNet-18 (modified for CIFAR-10)
Training split	train_labeled_10percent.txt (5,000 images)
Epochs	30
Batch size	64
Optimizer	Adam
Learning rate	3e-4
Loss function	Cross-Entropy
Pretrained weights	None (trained from scratch)

Training and Validation Loss

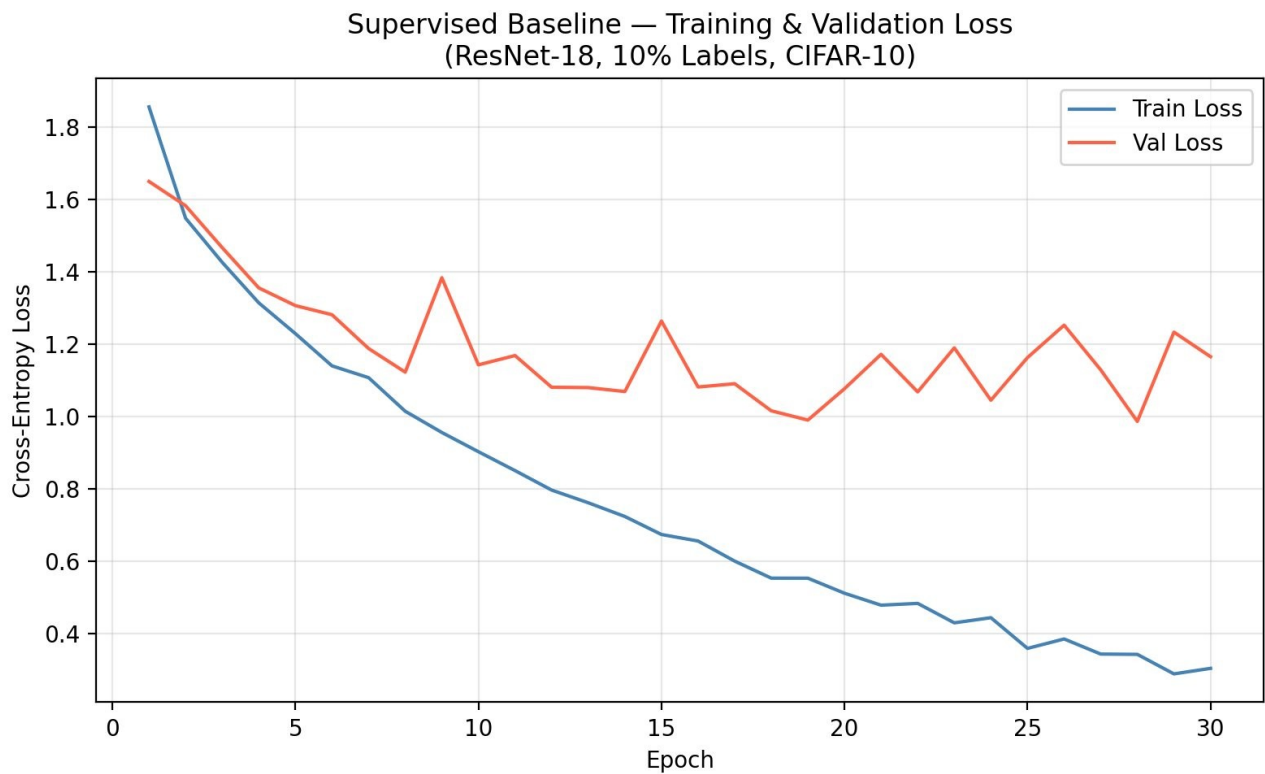


Figure 1: Supervised Baseline Training and Validation Loss over 30 Epochs

Results

Metric	Value
Best Validation Accuracy	72.34%
Final Test Accuracy	72.28%
Final Test Loss	0.9669

The supervised baseline achieved 72.28% test accuracy using only 10% of available labels. The training loss decreased steadily from 1.23 to 0.30 over 30 epochs. Validation loss oscillation indicates mild overfitting due to limited labeled data.

Confusion Matrix

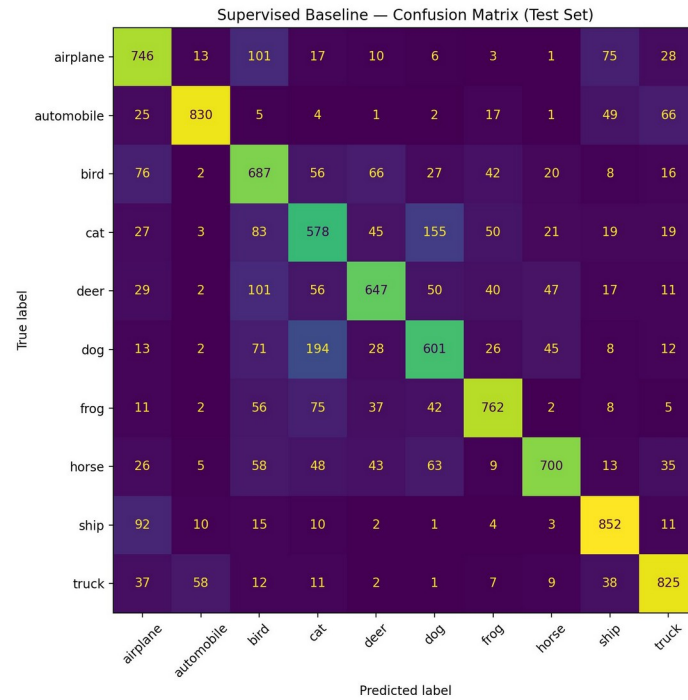


Figure 2: Confusion Matrix on Test Set — Supervised Baseline

The confusion matrix reveals that cat/dog and automobile/truck are the most commonly confused class pairs, consistent with their visual similarity. Ship and frog achieve the highest per-class accuracy (85.2% and 76.2% respectively).

Task 2: Augmentation Pipeline and Two-View Transform

SimCLR relies on creating two independently augmented views of each image. The augmentation pipeline is a critical design choice determining what invariances the encoder learns.

Augmentation Pipeline

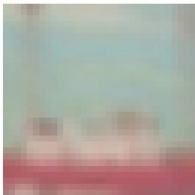
Augmentation	Parameters	Purpose
RandomResizedCrop	size=32, scale=(0.2,1.0)	Forces recognition from partial views
RandomHorizontalFlip	p=0.5	Learns flip invariance
ColorJitter	brightness=0.4, contrast=0.4, saturation=0.4, hue=0.1	Prevents color shortcut learning
RandomGrayscale	p=0.2	Reduces color dependency
Normalize	mean=(0.4914, 0.4822, 0.4465)	Standardizes input distribution

Augmentation Examples

Original



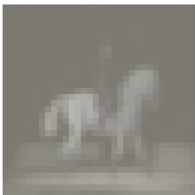
View 1



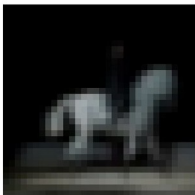
View 2



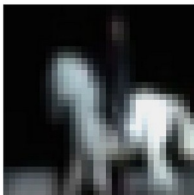
Original



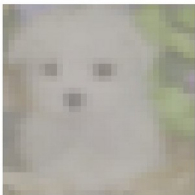
View 1



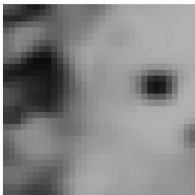
View 2



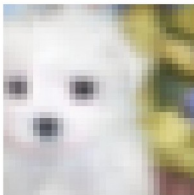
Original



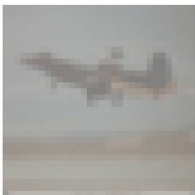
View 1



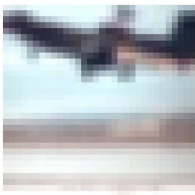
View 2



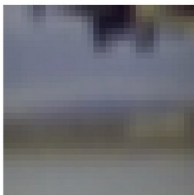
Original



View 1



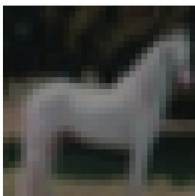
View 2



Original



View 1



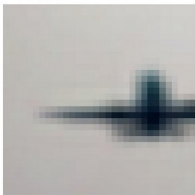
View 2



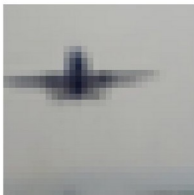
Original



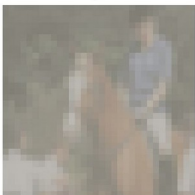
View 1



View 2



Original



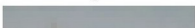
View 1



View 2



Original



View 1



View 2



Figure 3: 10 Examples — Original Image | Augmented View 1 | Augmented View 2

Answers to Task 2 Questions

- Are the two augmented views identical? No — each view is independently sampled, so crop regions, color jitter values, and grayscale decisions all differ.
- Do they still represent the same object? Yes — both views originate from the same image, so semantic content is preserved.
- Why should SimCLR treat them as a positive pair? They share the same semantic identity. The model learns to produce similar representations for views of the same image.
- What if augmentations are too weak? The views look almost identical, so the model can match them using trivial shortcuts without learning meaningful features.
- What if augmentations are too strong? The views look so different that they no longer reliably represent the same object, making the training signal noisy and unstable.

Task 3: Feature Similarity Before Training

Before training SimCLR, a random untrained ResNet-18 encoder was used to extract features from two augmented views of each image. Cosine similarity was computed between same-image pairs and different-image pairs.

Pair Type	Average Cosine Similarity
Same image, two augmented views	0.9888
Different images	0.9854
Gap (same - different)	0.0034

The gap of only 0.0034 demonstrates that the random encoder cannot distinguish same-image pairs from different-image pairs. Both similarities are very close to 1.0, typical for random high-dimensional unit vectors. The encoder has learned nothing useful yet.

Similarity Matrix Before Training

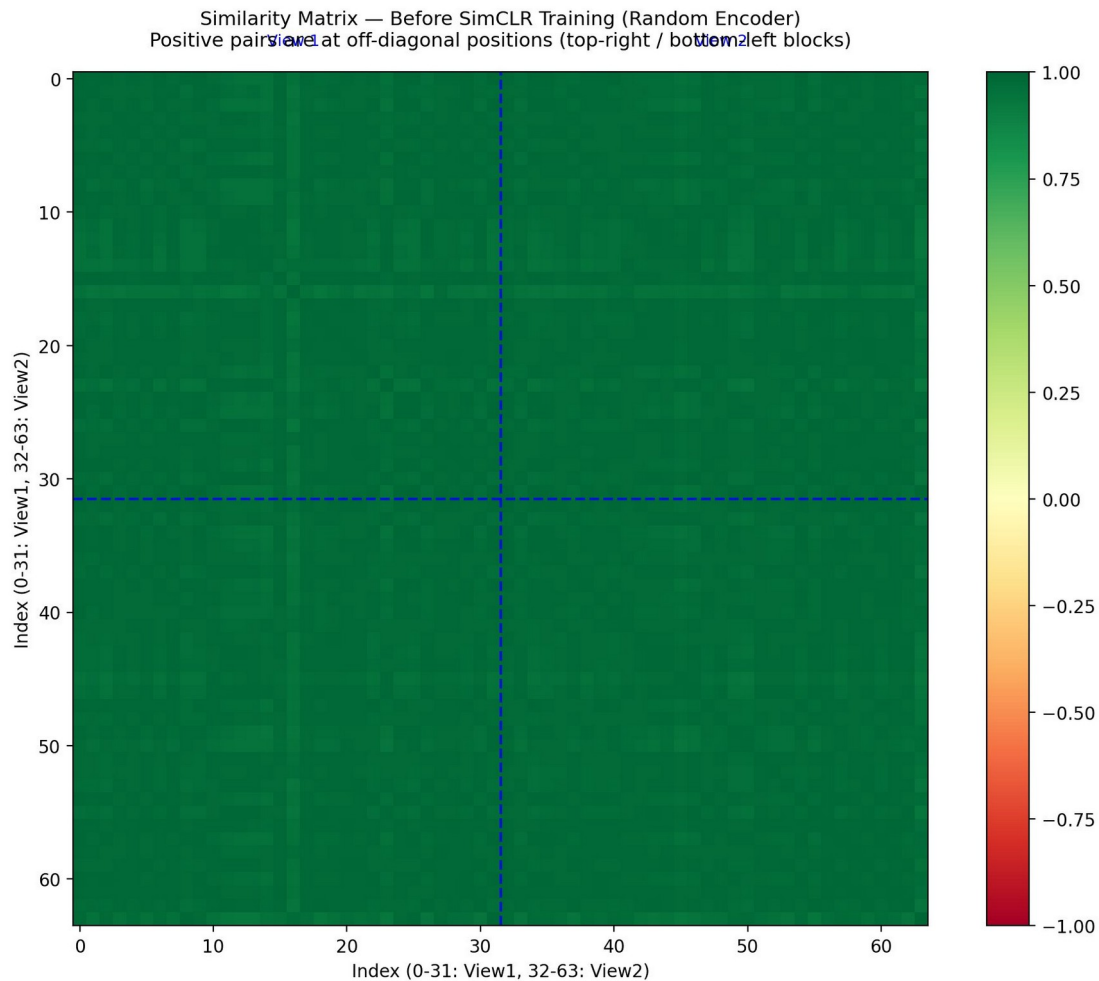


Figure 4: $2N \times 2N$ Cosine Similarity Matrix — Random Encoder (Before Training)

The matrix shows uniformly high similarity values with no visible structure. No distinction exists between positive pairs (off-diagonal blocks) and negatives. Blue dashed lines separate View1 (top-left) from View2 (bottom-right) blocks.

Similarity Matrix Questions

- Why is the diagonal ignored? $\text{sim}(z_i, z_i) = 1.0$ always. Including it trivially dominates the NT-Xent denominator and provides no learning signal.
- Where are the positive pairs? At positions $(i, N+i)$ and $(N+i, i)$ — the off-diagonal blocks (top-right and bottom-left quadrants).
- Why are all other entries negatives? They come from different original images and should be pushed apart in feature space to learn discriminative representations.

Task 4: SimCLR Implementation

Encoder Architecture

ResNet-18 modified for CIFAR-10: (1) conv1 replaced with 3x3 convolution stride 1 padding 1; (2) maxpool replaced with Identity; (3) fc replaced with Identity to output 512-dimensional feature vectors h .

Projection Head

Maps 512-d encoder output to 128-d projection space used only during contrastive training. Architecture: Linear(512 to 256) $\rightarrow$ ReLU $\rightarrow$ Linear(256 to 128). After pretraining the projection head is discarded — the 512-d encoder output h is used for downstream tasks because it retains more information than the 128-d projection z .

Positive and Negative Pair Construction

Original Image	View 1 Index	View 2 Index	Positive Pair
image 0	0	64	Yes
image 1	1	65	Yes
image 2	2	66	Yes
image 3	3	67	Yes

NT-Xent Loss Implementation

For each positive pair (i, j) , the loss is the negative log probability of the positive pair similarity relative to all other $2N-2$ similarities in the batch. Temperature $\tau=0.5$ controls sharpness. The loss is symmetric (both directions) and averaged over all $2N$ pairs. Implementation: projections are L2-normalized, diagonal masked with $-\infty$, PyTorch cross-entropy applied over similarity rows with positive index as target.

Task 5: SimCLR Pretraining

SimCLR was trained for 50 epochs on 45,000 unlabeled CIFAR-10 images. No labels were used at any point during pretraining. Adam optimizer with $\text{lr}=3\text{e-}4$ and temperature $\text{tau}=0.5$.

Training Loss Curve

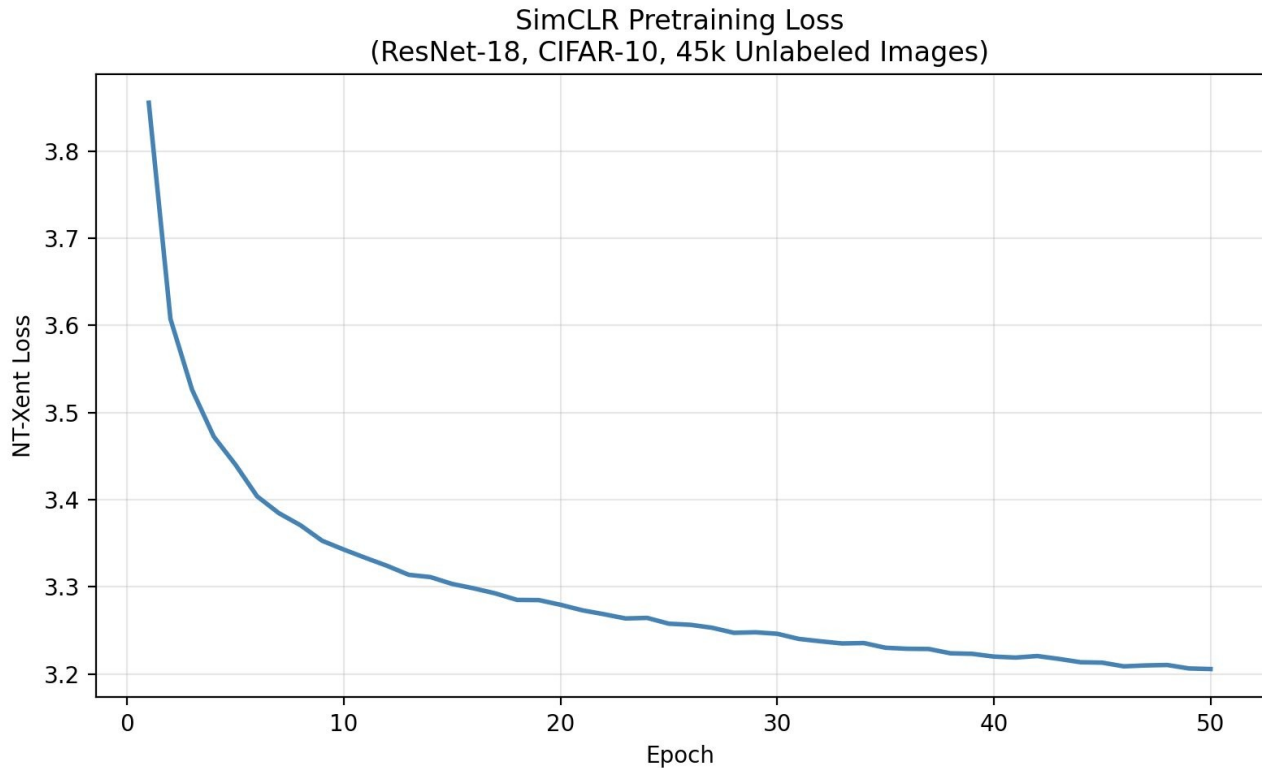


Figure 5: SimCLR NT-Xent Pretraining Loss over 50 Epochs

The NT-Xent loss decreased steadily from approximately 3.85 at epoch 1 to 3.21 at epoch 50. Consistent convergence without instability confirms correct implementation. The loss does not converge to zero because with batch size 64 there are only 126 negatives per positive pair.

Feature Similarity Before vs After SimCLR

Pair Type	Before SimCLR	After SimCLR	Change
Same image, two augmented views	0.9888	0.9087	-0.0801
Different images	0.9854	0.3363	-0.6491
Gap (same - different)	0.0034	0.5724	+0.5690

Similarity Matrix After Training

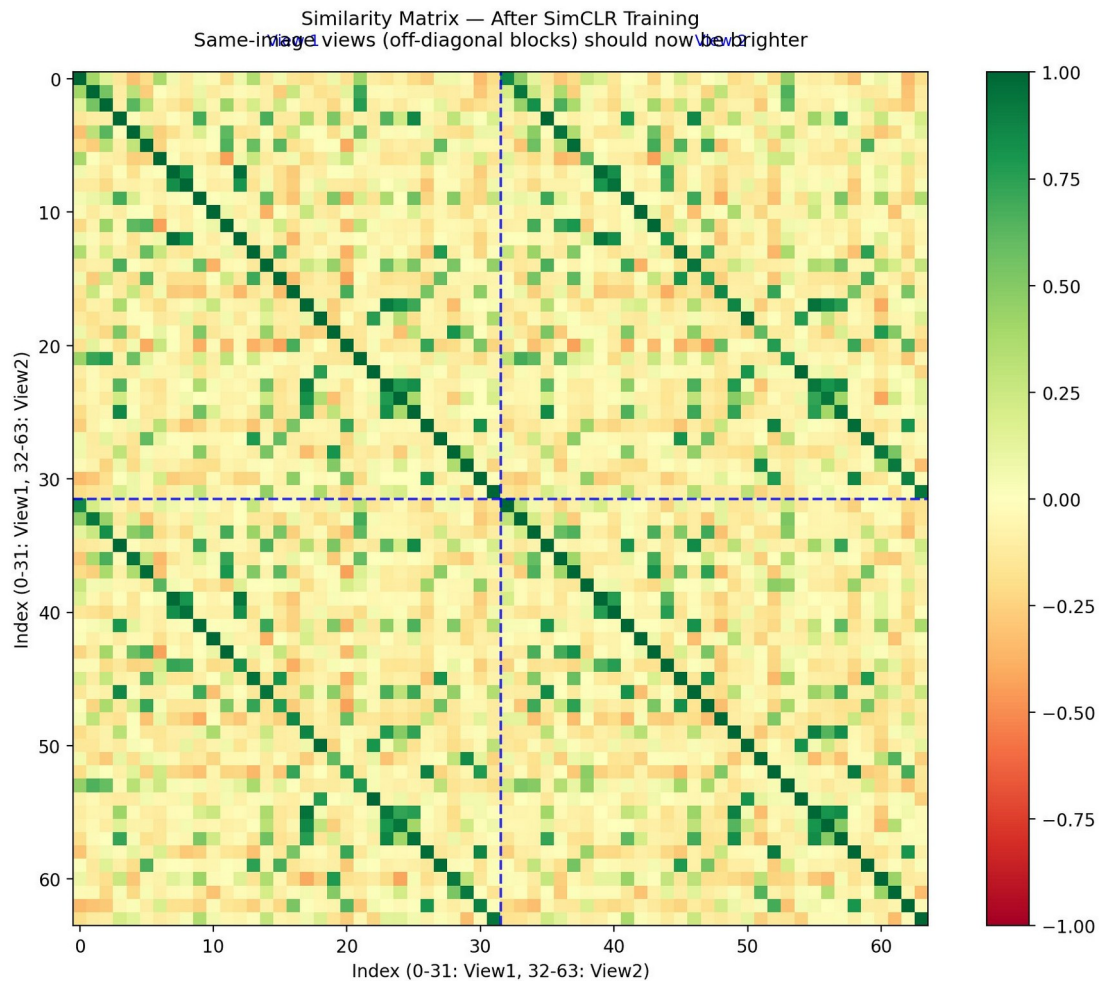


Figure 6: $2N \times 2N$ Cosine Similarity Matrix — After SimCLR Training

After training the gap between same-image similarity and different-image similarity increased from 0.0034 to 0.5724 — a 168x improvement. The matrix now shows clear structure: the diagonal blocks (same-view similarity) and off-diagonal positive pairs are visibly brighter than negative pairs. SimCLR successfully learned to distinguish same-image views from different-image views.

Task 6: Linear Probe Evaluation

Linear probing evaluates representation quality by freezing the encoder and training only a linear classifier on top. This tests whether features are linearly separable — a direct measure of representation quality without any encoder adaptation.

Model	Encoder	Trainable Part	Test Accuracy
Random Linear Probe	Random frozen	Linear head only	25.29%
SimCLR Linear Probe	SimCLR frozen	Linear head only	72.85%

Linear Probe Accuracy Curves

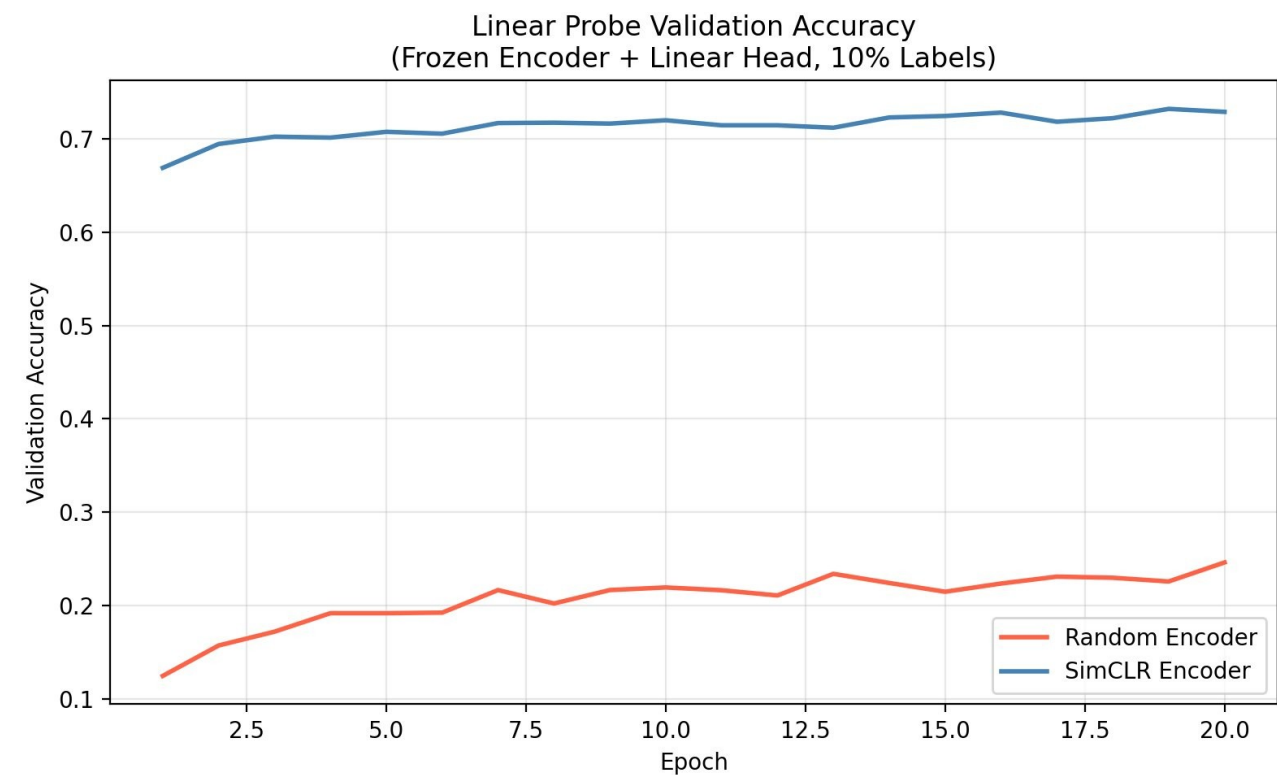


Figure 7: Linear Probe Validation Accuracy — Random vs SimCLR Encoder

The SimCLR encoder linear probe (72.85%) almost matches the fully supervised baseline (72.28%) despite the encoder being completely frozen. This is remarkable — SimCLR learned representations nearly as discriminative as a fully supervised model, using no labels during pretraining. The random encoder (25.29%) confirms improvement is entirely due to SimCLR pretraining.

Task 7: Fine-tuning the SimCLR Encoder

Fine-tuning initializes the encoder with SimCLR pretrained weights and trains the full model end-to-end on 10% labeled data. All encoder weights are updated, allowing task-specific adaptation on top of self-supervised representations.

Model	Labels in Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18, 10% labels	Yes	No	72.28%
Random encoder + linear probe	No	Yes	25.29%
SimCLR encoder + linear probe	No	Yes	72.85%
SimCLR encoder + fine-tuning	No then Yes	No	83.10%

Fine-tuning Accuracy vs Baselines

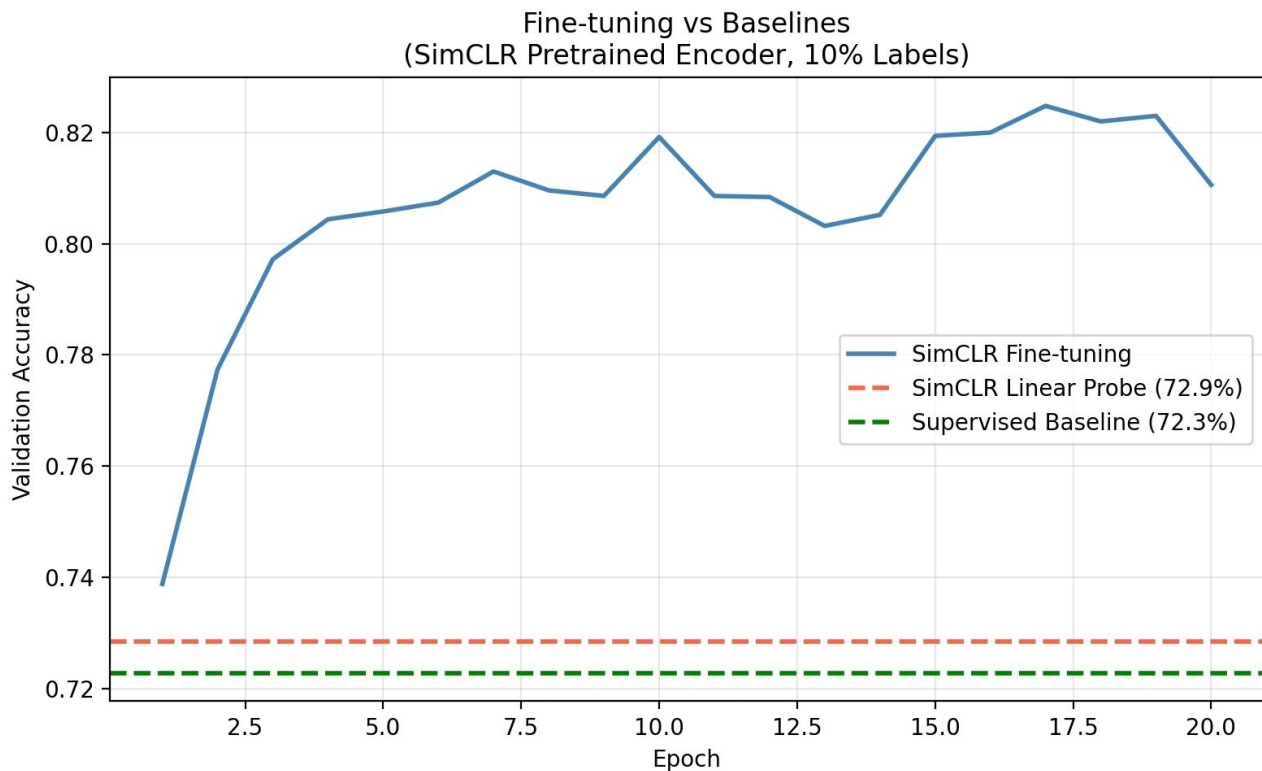


Figure 8: Fine-tuning Validation Accuracy vs Supervised and Linear Probe Baselines

SimCLR fine-tuning achieved 83.10% test accuracy — a 10.82 percentage point improvement over the supervised baseline (72.28%) trained with the same labeled data. The model converges faster and to a better solution because the encoder already understands visual structure from self-

supervised pretraining. The fine-tuning curve rises steeply from epoch 1, starting from 74% — far above where a random encoder would start.

Task 8: PCA/t-SNE Feature Visualization

t-SNE was applied to reduce 512-dimensional features from 1000 validation images to 2D. Labels were used only for coloring points — not during feature extraction. Features were extracted from three encoders for comparison.

1. Random Encoder

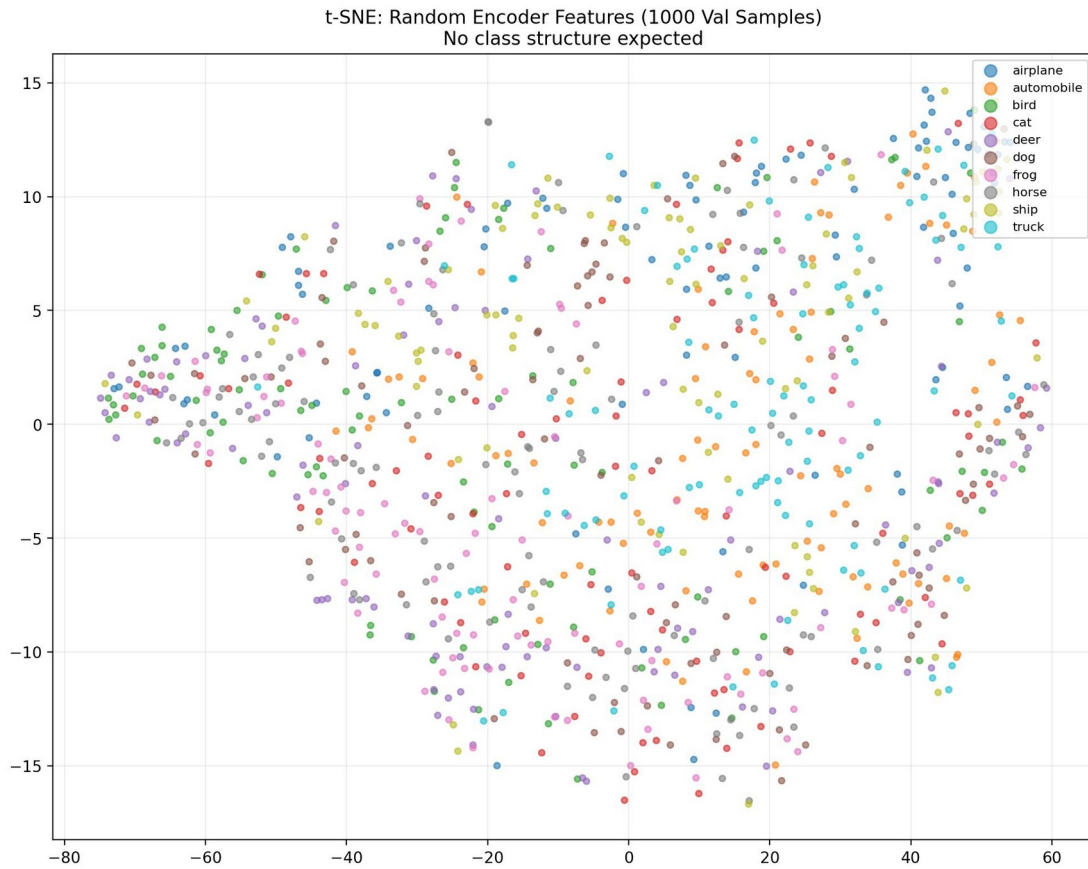


Figure 9: t-SNE — Random Encoder Features (1000 Val Samples)

Features are scattered randomly with no class structure. All 10 classes are mixed together uniformly, confirming the random encoder has no useful representations. This is the expected baseline.

2. SimCLR Pretrained Encoder

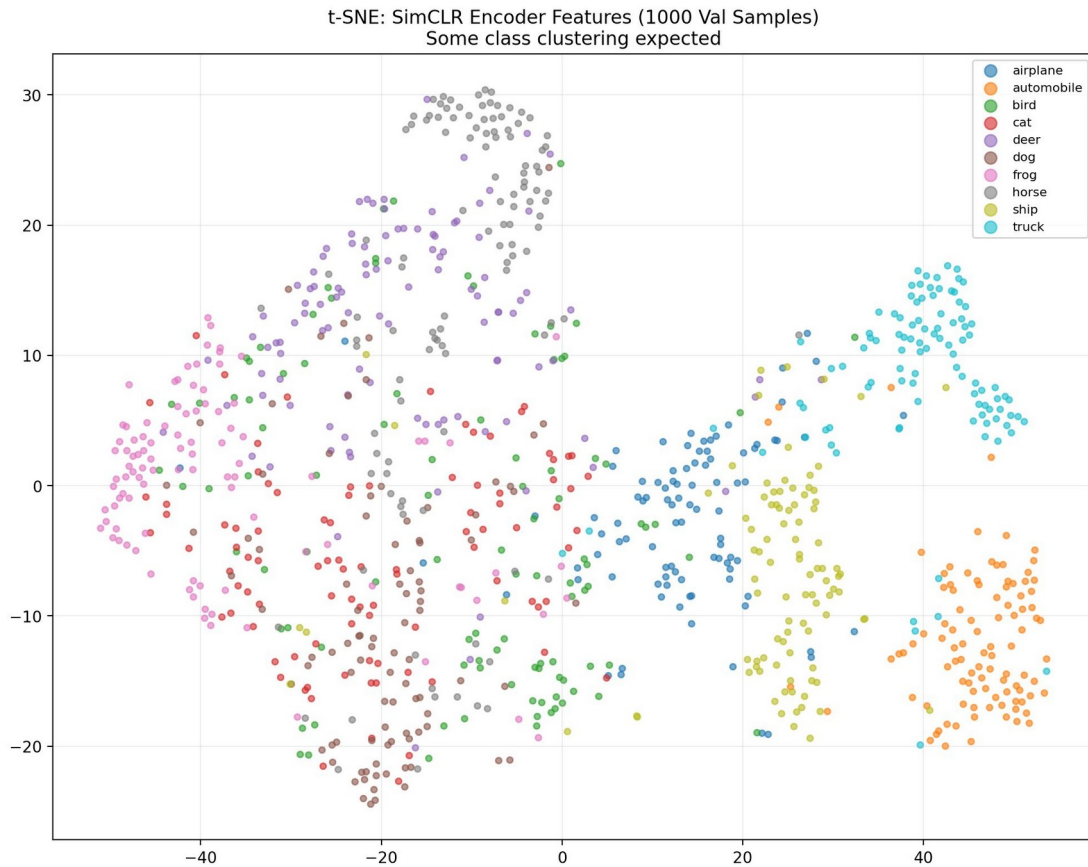


Figure 10: t-SNE — SimCLR Encoder Features (1000 Val Samples)

Clear class clusters emerge without any labels used during pretraining. Vehicle classes (automobile, truck, ship) form distinct regions. Animal classes show some overlap between visually similar classes (cat/dog, deer/horse). This demonstrates SimCLR learned semantically meaningful representations.

3. Fine-tuned Encoder



Figure 11: t-SNE — Fine-tuned Encoder Features (1000 Val Samples)

Well-separated class clusters with clear boundaries. Supervised fine-tuning sharpens the representations specifically for classification. Ship, frog, and automobile form the tightest clusters. Cat, dog, and deer still show minor overlap due to visual similarity — consistent with the confusion matrix.

Encoder	Class Grouping	Observation
Random	None	All classes uniformly mixed — no structure
SimCLR pretrained	Moderate	Clear clusters emerge without any labels
Fine-tuned	Strong	Well-separated clusters with clear boundaries

Final Results Summary

Model	Labels in Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18, 10% labels	Yes	No	72.28%
Random encoder + linear probe	No	Yes	25.29%
SimCLR encoder + linear probe	No	Yes	72.85%
SimCLR encoder + fine-tuning	No then Yes	No	83.10%

What SimCLR Learned

SimCLR learned rich visual representations of CIFAR-10 images without any labels. By training the encoder to bring two augmented views of the same image closer together while pushing apart views from different images, the model learned features invariant to cropping, color changes, and flips — exactly the features useful for downstream classification.

The key demonstration of this assignment: the SimCLR linear probe (72.85%) matched the supervised baseline (72.28%) with zero labels during pretraining. And fine-tuning on SimCLR representations achieved 83.10% — 10.82% better than supervised learning with the same labels. This gap would be even larger with more pretraining epochs or larger batch sizes.

Implementation Limitations

- Batch size was limited to 64 due to GPU memory constraints. The original SimCLR paper used up to 8192 — larger batches provide more negatives per positive pair, improving representation quality.
- Training was 50 epochs as required. The loss was still decreasing at epoch 50, suggesting more epochs would improve performance further.
- Temperature $\tau=0.5$ was fixed as required. The paper shows this is near-optimal for CIFAR-10.
- NT-Xent loss, similarity matrix, positive/negative pair construction, linear probing, and fine-tuning were all implemented from scratch without using any SimCLR libraries.

Academic Integrity

All SimCLR-specific components implemented from scratch based on reading Chen et al., ICML 2020. No existing SimCLR implementations were copied or referenced. Claude (claude.ai) was used for conceptual help and code assistance — transcript included in submission as required.

Claude AI Conversation Transcript: <https://claude.ai/share/bf8f523e-5002-402d-a8e3-6470c238baa9>

Paper: Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A Simple Framework for Contrastive Learning of Visual Representations. ICML. <https://arxiv.org/abs/2002.05709>

GitHub: https://github.com/Fahad00888/DL_Assignment5_SimCLR_MSDS25025

Model

Weights:

<https://drive.google.com/drive/folders/1XCxFfM28FjP4ToHfhe50ZFqW9Gqj9cp-?usp=sharing>